

9. MACsec Key Agreement protocol (MKA)

The MACsec Key Agreement (MKA) protocol allows PAEs (6.3), each associated with a Port that is an authenticated member of a secure connectivity association (CA) or a potential CA, to discover other PAEs attached to the same LAN, to confirm mutual possession of a CAK and hence to prove a past mutual authentication, to agree the secret keys (SAKs) used by MACsec for symmetric shared key cryptography, and to ensure that the data protected by MACsec has not been delayed.

MKA comprises a secure fully distributed multipoint-to-multipoint transport and a number of applications of that transport, including the distribution of SAKs by an elected key server using AES Key Wrap. The transport allows each PAE, possessing the secret key (CAK, 6.2) that is the token of its authentication and authorization to participate in a CA, to ensure that each received protocol message has been transmitted by another participant in current possession of that key and has not been delayed or replayed. The key distribution application ensures that no participant unwittingly reuses an SAK-nonce pair, even though that participant might have been powered-off, re-initialized, or making use of another key server as the consequence of temporary LAN partition. While both the transport and key distribution support group CAs, providing continuous secured connectivity even as the participants in the CA change, they impose little overhead where the number of group participants is two as compared to protocols designed solely for pairwise key agreement, and degenerate to well known forms in that common case.

In addition to distributing fresh SAKs, MKA manages their installation and use by the SecYs that secure the data transmitted and received by each Controlled Port, ensuring that each is capable of receiving data protected by that SAK before it is used for transmission. Each SecY uses MKA to communicate the lowest PN used for transmission with the SAK within the last two seconds, allowing receivers to bound transmission delays.

The Key Agreement Entity (KaY) responsible for MKA within the PAE can operate multiple simultaneous MKA instances: allowing CAK changes while maintaining secured connectivity; allowing confirmation of the hints provided by network discovery before a particular CA and CAK is selected; and allowing the key server to use a number of initial CAKs, each providing pairwise authentication between the server and a member of a group CA, to distribute the CAK for that group.

This clause specifies the following:

- a) Design requirements for MKA protocol functionality, including security considerations (9.1).
- b) Operational requirements that MKA places on the system and on network administration (9.2).
- c) The key hierarchy used by MKA (9.3).
- d) The secure transport provided by MKA to its applications (9.4).
- e) Key server election, using the secure transport to exchange information (9.5).
- f) How MKA capable PAEs decide to use, or not use, MACsec (9.6).
- g) Cipher suite selection (9.7).
- h) SAK generation, distribution, and adoption (9.8).
- i) Assignment of SAKs to SAs (9.9).
- j) SAK installation and use (9.10).
- k) Detection of changes in the secured connectivity provided by each Controlled Port (9.11).
- l) Group CAK distribution (9.12).
- m) Secure transmission of announcement and other parameters to provide confirmation of values previously received in unprotected EAPOL frames (9.13).
- n) Creation and deletion of MKA participants (9.14).
- o) Management of the KaY and MKA (9.16).
- p) Temporary suspension of MKA operation to facilitate in-service control plane software upgrades without disrupting existing secure connectivity (9.18).

The following terms are used to identify roles within the protocol or protocol scenarios:

- **participant:** The personification of a single KaY's participation in a given MKA instance (i.e., transmitting and receiving MKPDUs protected by keys derived from a single given CAK and identified by a given CKN), operating with positive intent, and obeying the protocol.
- **actor:** The participant under discussion, usually in the KaY being described.
- **partners:** Other participants in the same MKA instance, and attached to the same LAN, as the actor.
- **successful actor:** An actor that has one or more live partners and is participating in an MKA instance that has elected a Key Server.
- **principal actor:** The successful actor selected by a KaY to control its associated PAC or SecY.
- **member:** The personification of a single KaY's participation in all MKA instances and use of other controls that determine the connectivity provided by its associated Controlled Port. A system is described as being member of a CA if it includes a Controlled Port providing connectivity using that CA.

Throughout this standard 'random' and 'randomly chosen' numbers meet the criteria specified in 9.2.

9.1 Protocol design requirements

MKA meets the following requirements:

- a) It meets its security and other goals, detailed in these requirements, in the presence of an attacker that does not possess the CAK but can copy any frame sent by any participant, can selectively prevent delivery to some participants, and can transmit arbitrary frames to arbitrary participants.
- b) It provides and refreshes keys and other information that its clients (KaYs) use to establish, maintain, and manage secure connectivity in the presence of an attacker that does not possess the CAK but can copy frames transmitted by any participant, and can transmit arbitrary frames to arbitrary participants, up to the point that the load imposed by the reception of those frames exceeds the resources of a participant.
- c) Following any period of 8 seconds during which all frames transmitted by each of a set of participants are delivered, once and without misordering, to each of the other participants, and the load imposed by frames received from an attacker does not exceed the resources of any participant, MKA will provide the keys and information required by each of its participants' clients, irrespective of the prior state of each participant or frames buffered by the LAN.

NOTE 1—The requirement is that of correctness: the protocol convergence time is bounded; rather than a performance goal. The figure of 8 seconds arises from the possibility that prior participants are being timed out (over a period of MKA Life Time, 6 seconds, see Table 9-1) just as a new participant joins, plus MKA Hello Time (2 seconds) to ensure that all participants have subsequently transmitted.

NOTE 2—If MKA operation is suspended (9.18), and the participants do not already possess the necessary keys and information, convergence will be delayed until MKA operation resumes.

- d) It requires minimal transmission bandwidth, with a constant number of frames per second per participant in steady operation up to the MKA design goal of 30 participants.
- e) It does not require its participants to retain any information from prior instantiations of the protocol, with the exception of the CAK and CKN.
- f) It provides mutual discovery of the potential members of a CA, i.e., systems that possess the same CAK identified by the same CKN, that are attached to the same LAN.
- g) It provides proof of mutual liveness, i.e., mutual proof of current possession of the CAK, between all pairs of CA members.
- h) When used in combination with a SecY, with strict validation of received MACsec frames as specified by IEEE Std 802.1AE, it provides a reliable indication of the operPointToPointMAC status parameter.

- i) It detects duplication of the individual MAC addresses used to compose SCIs.
- j) It can be used to support mutual identification of participants that have previously completed mutual authentication, but are not capable of operating MACsec.
- k) It allows its participants to ensure that the data frames protected by MACsec are not being delayed by more than 2 seconds.

NOTE 3—Delay protection guards against an attack on the configuration protocols that MACsec is designed to protect by alternately delaying and delivering their PDUs. Delay protection does not operate if and when MKA operation is suspended (9.18).

MKA does not, is not capable of, or makes no attempt to defend against byzantine attacks, i.e., attacks made by systems that possess the CAK, including the following that are specifically identified as non-goals:

- l) Deliberate redistribution of a key by a participant acting as a key server.

MKA satisfies its security and convergence requirements to within the probabilities set by the use of pseudo random numbers generated as specified in 9.2.

When the option to support MACsec Cipher Suites that use Extended Packet Numbering is implemented, MKA meets the following requirements:

- m) Recovery, if lost, of the 32 most significant bits of the XPN, in support of the protocol correctness requirement c) above.

9.2 Protocol support requirements

A system that implements MKA shall satisfy this standard's requirements for the following:

- a) Random number generation
- b) SC identification

A system that implements MKA in support of MACsec shall also meet the protocol support requirements specified in IEEE Std 802.1AE.

9.2.1 Random number generation

The system shall provide a strong random number generator (RNG) for the generation of all numbers described as random in this standard. If a non-deterministic RNG (e.g., hardware RNG) is not available, the system shall make use of sufficient entropy to create a good quality seed for a deterministic RNG and should conform to the requirements of NIST Special Publication 800-108.

9.2.2 SC identification

Each SC is identified by an SCI that comprises a MAC address and a Port Identifier, unique within the system that has been allocated that address.

9.3 MKA key hierarchy

The root of key hierarchy for any given instance of MKA is the secure Connectivity Association Key (CAK), a secret key. Possession of a CAK for the CA is a prerequisite for membership in each CA supported by MACsec, and all potential members possess the same CAK and are attached to the same LAN.

The overall key hierarchy used by this standard is specified in 6.2, including the distribution of the CAK to, or its acquisition by, each potential member of a given CA and its relationship to authentication protocols. In summary, a CAK can be a direct result of a system's participation in EAP (Clause 8), a pre-shared key (PSK), or a key chosen by an MKA key server and distributed using a prior MKA instance. This subclause (9.3) specifies the following:

- a) How CAKs and derived keys are identified (9.3.1) within MKA.
- b) Independence of each CAK from its predecessors and successors (9.3.2).
- c) The derivation of further secret keys from the CAK (9.3.3) including the following:
 - 1) ICK, the ICV key used to verify the integrity of MKPDUs
 - 2) KEK, the key encrypting key used together with AES Key Wrap to distribute SAKs

9.3.1 CAK identification

Each CAK is identified by secure Connectivity Association Key Name (CKN) that allows each of the MKA participants to select which CAK, or CAK derived key, to use to process a received MKPDU. MKA places no restriction on the format of the CKN, save that it comprise an integral number of octets, between 1 and 32 (inclusive), and that all potential members of the CA use the same CKN. No further constraints are placed on the CKNs used with PSKs, though the network administrator should be aware that accidental use of the same CKN for different CAKs can inhibit communication and cause an operator to falsely conclude that a security attack is being attempted. CKNs for group CAKs that are generated by a Key Server shall include an RNG of at least 128 bits to guard against CKN collisions. See 6.2 for EAP specific CKN recommendations.

9.3.2 CAK Independence

A CA can be extremely long-lived, in infrastructure applications (see Clause 7) a design goal of tens of years without significant interruption in communication is not unrealistic, and it can be expected that systems supporting the CA (e.g., authentication servers) and even equipment directly participating in the CA will be changed during that time. MKA allows the communication between the members of a CA to be supported by a succession of MKA instances, as specified in 12.4, without interruption. While continuing communication while the CAK is changed necessarily implies that all of the systems that are members of the CA both before and after the change know both CAs, the two MKA instances each maintain their own CAK and derived keys quite separately, and neither instance discloses its CAK or derived keys for the other to its participants.

NOTE—Opinions differ as to how long an interruption in communication can last before it is considered significant, typically ranging from a few milliseconds to a few seconds. The goal of MKA, and its use in this standard, is to allow CAKs to be changed without any interruption to MACsec protected communication.

9.3.3 Derived keys

Each of the keys used by MKA is derived from the CAK. The CAK is not used directly. The derived keys are tied to the identity of the CAK, and thus restricted to use with that particular CAK.

To accommodate future developments in cryptography, each MKPDU conveys an Algorithm Agility parameter that identifies how the ICK is derived from the CAK, and how it is used. Each Algorithm Agility parameter value comprises four octets, the first three being those of an OUI (Organizationally Unique Identifier) or CID (Company ID) allocated by the IEEE Registration Authority, and the fourth allocated by the organization to which that OUI or CID has been allocated. The derivation of the ICK, and the use of this key as specified in this standard, is identified by the value specified in Table 9-1.

Table 9-1—MKA Algorithm Agility parameter values

Parameter value	Specification
00-80-C2-01	IEEE Std 802.1X-2020

The ICK is derived from the CAK using the KDF specified in 6.2.1. This KDF uses the AES Cipher in CMAC mode (IETF RFC 4493). The ICK is derived from the CAK using the following transform:

$$\text{ICK} = \text{KDF}(\text{Key}, \text{Label}, \text{Keyid}, \text{ICKLength})$$

where

Key = CAK

Label = “IEEE8021 ICK”

Keyid = the first 16 octets of the CKN, with null octets appended to pad to 16 octets if necessary

ICKLength = two octets representing an integer value (128 for a 128 bit ICK, 256 for a 256 bit ICK) with the most significant octet first

The Label is a UTF-8 string, without a null or other termination, exactly 12 bytes in length (the quotes shown do not form part of the string and exactly one space separates ‘8021’ and ‘ICK’). The length of the Label is chosen to make the input to the PRF within the KDF exactly 32 bytes. The hexadecimal representations of each of the text strings used with this KDF and test vectors are given in Annex G.

To accommodate future developments in cryptography, each of the MKA parameter set types used by MKA to communicate a wrapped key identifies the Key Wrap Algorithm and KEK derivation (6.2.4, Figure 11-11, Figure 11-12, Figure 11-13) used. The KEK used by all parameter set types currently specified in this standard is derived from the CAK using the KDF specified in 6.2.1. This KDF uses the AES Cipher in CMAC mode (IETF RFC 4493). The KEK is derived from the CAK using the following transform:

$$\text{KEK} = \text{KDF}(\text{Key}, \text{Label}, \text{Keyid}, \text{KEKLength})$$

where

Key = CAK

Label = “IEEE8021 KEK”

Keyid = the first 16 octets of the CKN, with null octets appended to pad to 16 octets if necessary

KEKLength = two octets representing an integer value (128 for a 128 bit KEK, 256 for a 256 bit KEK) with the most significant octet first

The Label is a UTF-8 string, without a null or other termination, exactly 12 bytes in length (the quotes shown do not form part of the string and exactly one space separates ‘8021’ and ‘KEK’). The length of the Label is chosen to make the input to the PRF within the KDF exactly 32 bytes.

When SAKs and CAKs are distributed they are protected by the KEK, using an AES Key Wrap as defined in 9.8.2 and 9.12.1.

SAKs should be derived from the CAK as specified in 9.8.1, but may also be generated directly by the Key Server’s strong random number generator (RNG, 9.2.1). Distributed CAKs, when used, shall be random values generated by the MKA Key Server RNG. Each distributed CAK is distinct from previously distributed CAKs, so that an MKA participant or attacker holding only the current CAK cannot determine a previously distributed CAK. This allows implementation of a policy of perfect forward security, with a fresh CAK being distributed when each participant joins a CA, so that participant cannot decrypt wrapped keys from previously transmitted MKA frames.

NOTE—MKA does not require fresh CAK distribution when a new participant joins a CA, as that would prolong the process of joining.

9.4 MKA transport

MKA provides a secure multipoint-to-multipoint transport between the members of the same CA, suitable for conveying information that is constant, or refreshed or acknowledged by the MKA applications that make use of that transport. The CAK is used to authenticate each protocol data unit (MKPDU) transmitted, providing proof of its transmission by a CA member, and each station includes its own randomly chosen identifier and a message number in the MKPDU. By transmitting MKPDUs that contain the identifiers and recent message numbers of the other participants, each member proves that it is in current possession of the CAK and is actively participating in the protocol, thus demonstrating the ‘liveness’ of the MKPDU and distinguishing it from MKPDUs that could have been captured by an attacker and played or replayed later—with the aim of disrupting the protocol or of influencing its outcome. MKPDUs are transmitted at regular intervals (see Table 9-3) of MKA Hello Time or MKA Bounded Hello Time (if a bounded receive delay is to be guaranteed, see 9.10), when data to be transported changes as specified in this clause (Clause 9), and as specified by the CP state machine (setting the state machine variable *newInfo*, as specified in Clause 12).

The message numbers also serve to enforce in-order delivery, and each of the MKA applications is designed so that the information conveyed in each MKPDU is idempotent, i.e., can be repeated without further changing the state of a recipient, and complete, i.e., fully expresses the desire of the transmitter for state change at the recipient. This design philosophy simplifies protocol analysis and allows a receiver to discard MKPDUs with prior message numbers.

The MKA transport is fully distributed and, as a consequence, robust in the face of the failure of any participant or of the LAN connectivity to that participant.

In principle the MKA transport could be used to communicate its participants’ desires to cooperate in a range of applications that require mutual authentication, and to state the relevant capabilities and requirements for each, since the MKPDU format and interoperable version rules permit the addition of parameter sets. In practice MKA is focused on MACsec. The generality provided by the MKA transport and MKPDU does permit future standardization of support for additional applications, alternatives for or additions to MACsec, and extensions and refinements for cipher suite negotiation. An example of MKA’s extensibility is provided by its use to secure announcements (9.13).

9.4.1 Message authentication

Each protocol data unit (MKPDU) transmitted is integrity protected by an 128 bit ICV, generated by AES-CMAC using the ICK (9.3):

$$\text{ICV} = \text{AES-CMAC}(\text{ICK}, \text{M}, 128)$$

$$\text{M} = \text{DA} + \text{SA} + (\text{MSDU} - \text{ICV})$$

In other words, M comprises the concatenation of the destination and source MAC addresses, each represented by a sequence of 6 octets in canonical format order, with the MSDU (MAC Service Data Unit) of the MKPDU including the allocated Ethertype, and up to but not including, the generated ICV.

NOTE—M comprises the whole of what is often referred to as ‘the frame’ considered from the point of view of the MAC Service provided by Common Port of the SecY (Figure 6-2) or PAC (Figure 6-6) supporting MKPDU transmission. The description does not use the term ‘frame’, because that Common Port could be supported by additional VLAN tags or other tags (consider the upper SecY shown in Figure 7-17) prior to transmission of a MAC frame by a system. Any such additional tags would not be covered by the ICV, and would be removed prior to MKPDU reception by a peer PAE.

Since the ICK is not directly distributed by any protocol, but only derived from the CAK, verification of the ICV both ensures that the contents of the MKPDU have not been modified but also that it was composed by a system that possessed the CAK.

Each MKPDU contains a secure Connectivity Association Key Name (CKN, 9.3.1) so intended recipients can identify the CAK, and hence the ICK, to be used to verify the ICV. The Algorithm Agility parameter conveyed in each MKPDU (9.3.3) permits future specification of other ICV algorithms and sizes.

9.4.2 Member identification and message numbers

Each participant in the protocol chooses a random (9.2) 96-bit member identifier (MI) when its participation in the protocol begins, and this MI is used, together with a 32-bit message number (MN) that is initialized to 1 and incremented with each MKPDU transmitted, to protect against delayed transmission or replay. Once a receiver has authenticated (9.4.1) an MKPDU with a given MI and MN, any MKPDUs with the same MI and the same or lower MN are discarded.

Use of the randomly chosen MI rather than a permanent unique identifier such as a MAC Address as the base for the anti-delay and anti-replay functions provided by the MN removes the need for a participant to maintain a record of the last used message number when powered off or re-initialized or for that participant to attempt to recover that data from its peers when an attacker has to be assumed to be active. Use of the MI also solves the issue of new participants understanding a potentially wrapping sequence number—when the MN reaches its upper limit the participant simply chooses a new MI and restarts the MN at 1.

While the MKPDUs themselves do not rely on the uniqueness of a key-nonce pair to ensure confidentiality or to avoid disclosing data that might lead to the recovery of a key, a MAC address forms part of the SCI which is a component of the nonce used by MACsec. Use of the MI makes it clear to any observer when, as rarely but occasionally happens, a duplicate MAC address is in use, either accidentally or as part of an attack on MACsec. The KaY will not enable MACsec transmission if there is any risk of a duplicate SCI.

The participant chooses a new MI if its current MI is already in use, as detected by either of the following:

- a) Its presence in the MI field of a received MKPDU.
- b) Its use in the Live Peer List or Potential Peer List of a received MKPDU in combination with an MN that is greater than that last transmitted by the participant.

9.4.3 Determining liveness

A participant proves liveness to each of its peers by including their MI, together with an acceptably recent MN, in an MKPDU with the participant's own MI and MN.

To avoid a new participant having to respond to each MKPDU from each partner as it is received, or trying to delay its reply until it is likely that MI.MN tuples have been received from all potential partners, each participant maintains and advertises both a Live Peer List and a Potential Peer List. The Live Peer List includes all the peers that have included the participant's MI and a recent MN in a recent MKPDU, and the Potential Peer List includes all the other peers that have transmitted an MKPDU that has been directly received by the participant or that were included in the Live Peer List of a MKPDU transmitted by a peer that has proved liveness. Peers are removed from each list when an interval of between MKA Life Time (see Table 9-3) and MKA Life Time plus MKA Hello Time has elapsed since the participant's recent MN (see above) was transmitted. This time is sufficient to ensure that two or more MKPDUs will have been lost or delayed prior to the incorrect removal of a live peer.

NOTE 1—The specified use of the Live and Potential Peer Lists thus permits rapid removal of participants that are no longer active or attached to the LAN while reducing the number of MKPDUs transmitted during group formation. For example, a new participant will be admitted to an established group after receiving, then transmitting, one MKPDU.

NOTE 2—A suspended participant (9.18) will be removed from the Live and Potential Peer Lists as described, but its associated SecY will still be able to transmit and receive secure frames until other CA members adopt a new SAK.

9.4.4 MKPDU information elements and application data

The MKA transport encodes the following information in each MKPDU:

- a) The CA Key Name
- b) The transmitting participant's SCI
- c) The transmitting participant's MI and MN
- d) The Live Peer List, comprising the MI and MN of each of the other participants believed to be in current possession of the CAK
- e) The Potential Peer List, comprising the MI and MN of each of the other potential participants

For encoding and decoding convenience, these elements are not consecutively encoded, in particular the live and potential peer lists are encoded after the information provided by the MKA applications. The details of MKPDU encoding and preliminary validation of received MKPDUs are specified in Clause 11.

9.4.5 Addressing

A LAN can be an individual LAN, bounded by the extent of its supporting media access method and media access method procedures, or can be supported by bridges in a Bridged Local Area Network or Virtual Bridged Local Area Network. Interoperability, avoiding redundancy, protecting the infrastructure, and the need to support communication between stations in the network, all necessitate placing restrictions on where, within the interface stack that composes a port (IEEE Std 802.1AC), SecYs are placed. Clause 11 of IEEE Std 802.1AE-2006 specifies the use of MACsec within systems, and those restrictions, but explicitly recognizes that MACsec can be used across, within, and to secure access to a Provider Bridged Network. The destination address used by MKA has to be aligned with the placement of port-based network access control in the interface stack and the corresponding use of MACsec within the protocol stack, and shall be a group address. Table 11-1 specifies group addresses that support the application scenarios described in this standard (Clause 7).

NOTE—Use of a group address that is filtered by bridges and the inclusion of destination and source MAC addresses within the ICV calculation makes attacking MKA from a distance more difficult.

The source address of each MKPDU shall be an individual MAC Address assigned to the port transmitting that MKPDU.

9.4.6 Active and passive participants

A participant can be active, transmitting periodic MKPDUs, or passive. A passive participant will become active for a period of MKA Lifetime following the receipt of an MKPDU from a feasible partner, i.e., provided that either the receiving participant or the partner is prepared to act as a Key Server. Whether a participant is to be active when first created, and whether it is to remain active in the absence of feasible partners depends on the port-based network access control application. The creation of passive participants supports systems that have many potential peers, with only one or a few likely to be connected at a time. Participants that are always active are desirable where connectivity is provided by media that do not reliably signal loss and resumption of connectivity, as can be the case for infrastructure links supported by virtual media. If all the participants in a potential CA can be passive, and an extended and undetected network outage occurs, it is possible that the potential CA members will fail to transmit MKPDUs, resulting in a permanent lack of connectivity.

NOTE 1—The condition of having recently received an MKPDU from a feasible partner can be determined by inspecting the participant's Live Peer List and Potential Peer List.

NOTE 2—The model of Logon Process operation encompasses participant creation, deletion, and control of active or passive participation (12.2, 12.5.2). The CAK cache provides management (see **activate** in 9.16).

An active participant will remain active, even in the absence of received MKPDUs, while a suspension is in progress (provided that the participant is not itself suspended). A suspended participant will resume active operation if and only if

- a) Its CAK was previously cached, and
- b) The management controls associated with that cached CAK specify that it is to be active on resumption (9.16).

9.5 Key server election

The participants in a given MKA instance agree on a Key Server, responsible for the following:

- a) Deciding on the use of MACsec (9.6)
- b) Cipher suite selection (9.7)
- c) SAK generation and distribution (9.8)
- d) SA assignment (9.9)
- e) Identifying the CA when two or more CAs merge (9.11)
- f) CA formation and group CAK distribution (9.12)
- g) Initiating, continuing, and terminating MKA suspension (9.18)

If the CAK is a pairwise CAK derived directly from EAP (see 6.2.2), the MKA participant for the PAE that was the EAP Authenticator will be the Key Server, and will not accept information [a) through g) above] from any other participant that attempts to act as the Key Server for that MKA instance. Each participant in an MKA instance with a CAK that was not directly derived from EAP uses the Key Server Priority (an 8-bit integer) encoded in each MKPDU to agree on the Key Server. Each such participant selects the live participant advertising the highest priority as its Key Server whenever the Live Peer List changes, provided that highest priority participant has not selected another as its Key Server or is unwilling to act as the Key Server [as indicated by 9.5.1 b)]. If a Key Server cannot be selected SAKs are not distributed. In the event of a tie for highest priority Key Server, the member with the highest priority SCI is chosen. For consistency with other uses of the SCI's MAC Address component as a priority, numerically lower values of the Key Server Priority and SCI are accorded the highest priority. Table 9-2 contains recommendations for the use of priority values for various system roles. Participants that will never act as a Key Server should advertise priority 0xFF.

Table 9-2—Key Server Priority values

System role of network access controlled port	Recommended or default value	Recommended range
Infrastructure port	0x10	0x00 – 0x1F
Primary Network Access Point	0x30	0x20 – 0x3F
Secondary or backup Network Access Point	0x50	0x40 – 0x5F
Group CA member	0x70	0x60 – 0x7F

As CA membership changes and before all current participants recognize each other as live, more than one participant can decide that it is the Key Server and attempt to fulfill the Key Server's responsibilities. The design of the SAK distribution and other applications where the Key Server plays a role is such that each member can simply accept data from the Key Server that member has elected.

If a KaY participates in multiple MKA instances so that there are several actors—one per instance—for a given port, then only the actor selected as the principal actor (12.1) will (if elected Key Server for its CAK) distribute SAKs. Since the factors that cause a principal actor to be selected from its peers are the same for different CAK, CKN tuples with the same distribution, replacement of a principal actor by its successor will

occur, whenever possible, without a change of Key Server. This succession plan ensures that the SAKs distributed using one CAK, CKN tuple can be followed immediately by SAKs distributed by a successor CAK, CKN without any loss of MACsec connectivity. To minimize the chance of a CA member that possesses both CAKs temporarily losing connectivity, a Key Server should not distribute an SAK using the new CAK until MKA Life Time (Table 9-3) has elapsed after it has started participating with that CAK, and should not delete the participant for the prior CAK until MKA Life Time has elapsed after that new SAK is first distributed. The CP state machine (Figure 12-2) ensures that a new SAK is not distributed until the Key Server is receiving and transmitting using a single SAK.

NOTE 1—If two PAEs are each capable of acting as both an EAP Authenticator and an EAP Supplicant, their interaction can result in two instances of successful mutual authentication with each acting as the Authenticator in one. Thus two MKA instances can be created, and each PAE's KaY will be elected Key Server for one instance. However, each KaY will select, as its principal actor, its participant in the MKA instance with the highest priority Key Server. That Key Server can then use that MKA instance to distribute SAKs (or select unauthenticated connectivity, see 12.3), and the participants in the other MKA instance can be deleted.

NOTE 2—A number of KaYs participating in multiple MKA instances will still succeed in configuring a single CA even if each participates in a different subset of those instances, provided that the highest priority Key Server in each subset itself participates in an MKA instance with a higher priority Key Server or is the highest priority Key Server. For example, assume KaYs A, B, C, D (say) in decreasing priority order, with two MKA instances with participants {A, B, C} and {B, D} respectively. Although B will be elected Key Server for the {B, D} instance, it will not distribute SAKs to D, as B's principal actor will be in {A, B, C}. Thus a single CA will be created, including the ports associated with A, B, and C, but excluding D's. However, scenarios of this type are most likely to result from errors in manual key distribution. They can give rise to temporary interruptions or unwanted connectivity, particularly where in-service upgrades are to be performed and need to be eliminated as part of managing upgrades (9.18.6).

9.5.1 MKPDU application data

Each MKA participant encodes the following information in every MKPDU transmitted:

- a) Key Server Priority, the priority of the transmitting participant.
- b) Key Server, a flag set if and only if the participant has not decided that another participant is or will be the Key Server.

9.6 Use of MACsec

Each participant advertises whether it is capable of using MACsec, and whether it requests MACsec protection of any communication through the Controlled Port of a SecY. The Key Server decides whether all, or none, of the participants in the CA should use MACsec to protect transmit frames; thus guarding against partial connectivity within the CA.

NOTE 1—The 'MACsec desired' parameter is provided to allow a network administrator to deploy MACsec capable systems, to verify their operation—including their participation in authentication and key agreement protocols, and to ensure that network operation does not depend on communication with systems that are not MACsec capable, before MKA selects MACsec protection and restricts the CA to MACsec capable systems.

To avoid duplication or conflict with the management controls for the SecY's secure frame generation and verification (10.5 and 10.6 of IEEE Std 802.1AE-2018) direct management access to the protectFrames parameter is disabled. If the Key Server and at least one other live participant are MACsec capable, and at least one of those participants requests MACsec protection, the Key Server advertises 'MACsec protect' and each participant sets its SecY's protectFrames parameter. Otherwise, protectFrames is cleared.

NOTE 2—IEEE Std 802.1AE mandates the capability to disable management access to individual parameters.

Participants that receive or transmit unprotected frames are responsible for ensuring that their protocol clients select appropriate policies. Whether any communication is desirable depends on the system configuration (see 7.3.3). If the Key Server selects MACsec protection, any MKA participant that is not capable of using MACsec (or the selected cipher suite) will not receive or transmit using the Controlled Port, and will maintain MAC_Operational False for that port.

9.6.1 MKPDU application data

Each MKA participant encodes the following information in every MKPDU transmitted:

- a) MACsec capability, indicating whether MACsec is implemented, and if so whether the implementation provides integrity protection only, integrity and integrity with confidentiality, or integrity and integrity with confidentiality with a selectable confidentiality offset of 0, 30, or 50 octets (see Table 11-6, IEEE Std 802.1AE).

NOTE—IEEE Std 802.1AE-2006 introduced the confidentiality offset to facilitate early MACsec deployment on existing systems that needed to store received frames before applying MACsec processing and that needed to examine the initial octets of received frames to decide where to store those frames. The XPN Cipher Suites standardized in IEEE Std 802.1AE do not support confidentiality offsets of other than 0.

- b) MACsec desired, a flag, set if the participant desires the use of MACsec to protect frames.

9.7 Cipher suite selection

The cipher suite, if MACsec is used to protect frames, is selected by the Key Server and advertised with each key.

Cipher suites are identified by a reference number (Table 14-1 of IEEE Std 802.1AE-2018) comprising eight octets, assigned by an organization that has been authorized to use a 24-bit OUI (IEEE Std 802) or a 36-bit OUI-36 by the IEEE Registration Authority. If the assignment is OUI based, the first three octets of the reference number are the octets of the OUI with the least significant bit of the first octet zero. If the assignment is OUI-36 based, the first four octets and the four most significant bits of the fifth octet comprise the OUI-36. The remainder of the reference number is assigned by the organization. The reference number should not correspond to any other EUI-64 assignment made by the assignee, nor should the first six octets be those of any EUI-48 assignment.¹⁸

An MKA Key Server can also elect not to use MACsec to secure communication, using MKA simply to prove prior authentication (see 7.2). The Key Server uses an EAPOL-MKPDU to signal such use by including the Distributed SAK parameter set with a zero parameter set body length (see Figure 11-13). It is often convenient to refer to the use of plain text transmission in this context as use of the Null Cipher Suite. This standard allocates the cipher suite reference number FF-FF-FF-FF-FF-FF-FF-FF to identify plain text transmission.

9.7.1 MKPDU application data

A participant that believes itself to be the Key Server and its KaY's principal actor encodes the following information with each MACsec SAK that it distributes, unless the mandatory Default Cipher Suite GCM-AES-128 is to be used:

- a) MACsec Cipher Suite, the Cipher Suite reference number.

The following information is also distributed with each MACsec SAK:

- b) Confidentiality Offset, indicating whether confidentiality is to be provided, and whether an offset of 0, 30, or 50 octets is used (see IEEE Std 802.1AE).

NOTE—The XPN Cipher Suites standardized in IEEE Std 802.1AE do not support confidentiality offsets of other than 0.

If a receiving MKA participant does not implement the referenced Cipher Suite with the selected confidentiality offset, the distributed SAK will not be installed (12.4). An MKA participant should advertise

¹⁸See the "Guidelines for Use of Extended Unique Identifier (EUI), Organizationally Unique Identifier (OUI), and Company ID (CID)."

any Cipher Suites implemented in addition to the Default Cipher Suite by including an Announcement parameter set (11.11.1, Figure 11-15) with a MACsec Cipher Suites TLV (11.12.3) in each MKPDU transmitted.

9.8 SAK generation, distribution, and selection

The Key Server is responsible for generating (9.8.1) and distributing MACsec SAKs, using AES Key Wrap (9.8.2), to each of the other members of the CA, using the MKA transport.

Each SAK is identified by a 128-bit Key Identifier (KI), comprising the Key Server's MI (providing the more significant bits) and a 32-bit Key Number (KN) assigned by that Key Server (sequentially, beginning with 1). Each KI is used to identify the corresponding SAK for the purposes of SAI assignment (9.9), and appears in the clear in MKPDUs, so network management equipment and personnel can observe and diagnose MKA operation (if necessary) without having access to any secret key.

NOTE 1—A KN of 0 indicates that MACsec is not being used by the transmitting participant. Use of this reserved KN value allows the participant to report that it has elected the Key Server whose MI appears in the KI.

If the Current Cipher Suite is not using extended packet numbering, the Key Server observes the Key Identifier and Lowest Acceptable PN for the most recent SAK in use, as transmitted by each CA member (the LKI and LLPN if LRX is true, and the OKI and OLPN otherwise; 9.10.1), and shall distribute a fresh SAK (subject to the constraints specified in 9.5 and this clause) if that Key Identifier matches the KI of the key most recently distributed and that Lowest Acceptable PN equals or exceeds the constant PendingPNExhaustion. PendingPNExhaustion is 0xC000 0000 for 32-bit PNs and 0xC000 0000 0000 0000 for 64-bit PNs. Subject to conditions [a) through c), below] that limit the frequency of SAK changes, postponing their generation and distribution until CA membership is likely to be stable, the Key Server shall also distribute a fresh SAK whenever a member is added to the live membership of CA (as perceived by the Key Server—with each MI, not the associated SCI, representing each member), and can distribute a fresh SAK when a member is removed from the live membership. If the Key Server has not generated a fresh SAK since the last addition to the Key Server's Live Peer List, an SAK is not distributed in transmitted MKPDUs. A fresh SAK is not distributed until:

- a) The Key Server's Live Peer List contains at least one peer, and
- b) An MKA suspension is not in progress, i.e., the Key Server either does not support suspension (5.11.4), or the Key Server's `suspendedWhile` timer is zero (9.18), and
- c) Either
 - 1) MKA Life Time (Table 9-3) has elapsed since the prior SAK was first distributed, or
 - 2) The Key Server's Potential Peer List is empty.

Once a Key Server has generated an SAK, it shall be distributed in each MKPDU transmitted by its principal actor until all live peers that can use the selected Cipher Suite and Cipher Suite capability (9.6.1) report having installed the SAK for receive or until a change in the live membership of the CA requires the generation of a fresh SAK.

NOTE 2—The CP state machine (Figure 12-2) ensures that SAKs are not continually changed faster than they can be installed and used by each of the CA members.

An MKA participant does not retain any record of SAKs used prior to initialization or re-initialization, and uses a fresh MI whenever it is initialized, thus forcing distribution of a fresh SAK whenever it has no record of prior SAK use. An MKA participant is re-initialized, or deleted and a fresh participant created, if the associated SCI is changed. An MKA participant accepts only SAKs distributed by Key Servers that are mutually live, i.e., shall not accept an SAK distributed in any MKPDU that does not contain that participant's MI and acceptably recent MN in the Live Peer List.

NOTE 3—Inclusion in the Potential Peer List is sufficient to prove that the Key Server is live, but not that it has recognized the participant as live and updated the distributed key. Reinitializing and using a new MI if an SCI is changed prevents use of the new SCI in conjunction with a previously distributed SAK, allows the Key Server to check for SCI collision before distributing a fresh SAK, and means that the Key Server and other participants are aware of the change when calculating SSCI values.

Each participant shall record the values of KI, nextPN for secure frame generation (see IEEE Std 802.1AE), for the last SAK accepted from each Key Server (as identified by the MI that forms part of the KI), to ensure that the SAK-nonce pair is not reused if the member switches (or is induced to switch) to a different Key Server and back to a previously used Key Server. Each member shall be capable of recording at least four such KI, nextPN tuples, and shall reinitialize (thus choosing a new MI, and forcing distribution of a fresh SAK) if the number of Key Servers used since the MI was chosen exceeds its ability to maintain all the relevant tuples, irrespective of the time elapsed or perceived to have elapsed since the SAK distributed by a given Key Server was last used.

Once a Key Server has distributed an SAK, it shall not distribute any previously generated SAK in any subsequent MKPDU. However, it is recognized that successive SAKs will have the same value with a probability of no less than 1 in $2^{-\text{keysize}}$ when generated as specified (see 9.8.1).

A Key Server shall distribute a fresh SAK whenever it selects a new MACsec Cipher Suite (9.7) unless the new Cipher Suite is the Null Cipher Suite, i.e., plain text transmission is selected. The CP state machine (12.4) ensures that this requirement is met.

9.8.1 SAK generation

Each SAK should be generated using the KDF specified in 6.2.1 using the following transform:

$$\text{SAK} = \text{KDF}(\text{Key}, \text{Label}, \text{KS-nonce} \mid \text{MI-value list} \mid \text{KN}, \text{SAKlength})$$

where

Key	=	CAK
Label	=	“IEEE8021 SAK”
KS-nonce	=	a nonce of the same size as the required SAK, obtained from an RNG each time an SAK is generated.
MI-value list	=	a concatenation of MI values (in no particular order) from all live participants
KN	=	four octets, the Key Number assigned by the Key Server as part of the KI
SAKlength	=	two octets representing an integer value (128 for a 128 bit SAK, 256 for a 256 bit SAK) with the most significant octet first.

‘|’ denotes concatenation of octet strings.

NOTE 1—The CAK and KS-nonce are both secret values that contribute cryptographic entropy to the SAK. While the randomly generated MI-values and the KN are not secret, their addition allows further permutation of the keying material, contributing freshness to the key generation process. Including the KN ensures that this contribution will not repeat unless the Key Server repeats its MI choice, while all other participants repeat or maintain their MI choices. In many deployment scenarios a restart will change MAC_Operational for two connected participants, causing both to reselect their MI.

NOTE 2—Specifying an MI-value list order would be of no value, as key generation is local to the Key Server.

Alternatively, SAKs may be generated directly by the Key Server’s RNG (9.2.1).

9.8.2 Use of AES Key Wrap

Each distributed SAK shall be protected by AES Key Wrap, as specified by IETF RFC 3394. The AES Key Wrap default IV defined in that specification shall be used.

9.8.3 MKPDU application data

Each participant that considers itself to be the current Key Server can distribute an SAK by encoding the following information in transmitted MKPDUs:

- a) Distributed SAK, the SAK, protected by AES Key Wrap (see 6.2.4, 9.8.2, Figure 11-11).
- b) The KN, 32 bits.

Only one Distributed Key is present in an MKPDU, although each participant can be using up to two SAKs to protect MACsec data (9.10).

9.9 SA assignment

The Key Server identifies the association number (AN) to be used with each key it distributes, using ANs in sequence (0,1,2,3,0,...) unless it yields to a higher priority key server for a period, and beginning the sequence with the first AN following the last SAK in use by any live CA member when the SAK was first distributed.

9.9.1 MKPDU application data

The Key Server encodes the following information in every MKPDU transmitted that includes a Distributed SAK, i.e., that has the Key Present flag set:

- a) Distributed AN, an enumeration [0,1,2,3]

9.10 SAK installation and use

Each CA member's KaY uses the LMI supported by the SecY (10.7 of IEEE Std 802.1AE-2018) to create a receive SC for each of its live peers' SCs and a SA for each receive SC and its own transmit SC(s) using the distributed SAK and AN.

NOTE 1—As specified in IEEE Std 802.1AE, an MKA participant is associated with each transmit SC even if multiple transmit SCs (each with its own SCI) are associated with one KaY. Only one of those participants can act as the Key Server at a time. MIs for the others are included in its Live Peer List.

Each participant advertises the status of the receive SAs and its transmit SA. The Key Server will enable transmission for its transmit SA, immediately if it was not previously transmitting or receiving, and when all live peers report that they can receive using the corresponding SAK and AN otherwise. The other participants enable transmission when they see that the Key Server is transmitting using the SAK and AN. See the CP state machine (12.4, Figure 12-2).

If the KaY comprises more than one actor, only SAKs that are received by the principal actor (or distributed by that actor if it is the Key Server itself) are installed.

Each CA member advertises the lowest PN used for each SAK (LLPN, OLPN—Lowest Acceptable PN for the Old Key) within one second prior to transmitting each MKPDU window, and may use the time bounds provided by received LPNs and the reflection of its own MI.MN tuples in the Live or Potential Peer List to discard delayed traffic.

NOTE 2—Enforcement of bounded received delay necessitates transmission of MKPDUs at frequent (0.5 s) intervals, to meet a maximum data delay of 2 s while minimizing connectivity interruption due to the possibility of lost or delayed MKPDUs. MKPDU can therefore operate without data delay protection, to lessen its processing requirements.

When MKA is used in conjunction with an XPN Cipher Suite, an SSCI is required for each SA. As specified in IEEE Std 802.1AE, the SA with the numerically greatest SCI uses the SSCI value 0x00000001, that with the next to the greatest SCI uses the SSCI value 0x00000002, and so on. The value 0x00000000 is not used.

PAEs that implement MKA Version 3 (or higher) order the MIs in the Live Peer List of transmitted MKPDUs. The MI associated with the numerically greatest SCI occurs first in the list, that with the next to greatest SCI second, and so on. The Key Server's own MI is not included in the Live Peer List. The SSID of the Key Server is also encoded in each MKPDU used to distribute an SAK for an XPN Cipher Suite, and indicates not only the value to be used by the Key Server, but also is corresponding to the position that it would otherwise occupy in the list. This information allows a receiving participant to determine the SCI to SSCI mappings for the transmit SAs used by it and each of the participants in the MKPDU's Live Peer List that are also in its own Live Peer List, even if those lists differ as a result of MKPDU loss or delay.

On receipt of a Version 3 or higher MKPDU distributing an SAK for an XPN Cipher Suite, a PAE implementing MKA Version 3 determines SSCI values as follows. The Key Server's SA takes the SSCI value explicitly encoded in the Live Peer List in the MKPDU. The SSCIs are then assigned in order to each of the SAs. The MI values that have a position ordinal lower than the Key Server's SSCI are given their ordinal value. The MI values that have a position in the Live Peer List of the Key Server's SSCI or greater are given an SSCI of their ordinal value + 1. So, for example, if the Key Server's transmit SA's SSCI is 0x00000002 and there are three MI values in the MKPDU Live Peer List, then their transmit SA's SSCIs are 0x00000001, 0x00000003, and 0x00000004, respectively. A receive SA is not created for any MI that is not in the Live List of a participant receiving a distributed SAK, but the ordered Live List from the MKPDU distributing that SAK is retained so the receive SA can be created when an MKPDU with that MI, proving liveness and conveying an SCI, is received.

NOTE 3—Since SSCIs are assigned sequentially and MKPDUs can convey information for only a limited number of participants, only the least significant octet of the Key Server's SSCI is encoded in MKPDUs; each of the more significant octets has the value 0x00.

9.10.1 MKPDU application data

Each CA member encodes the following information in every MKPDU transmitted, for the latest and the old AN:

- a) AN
- b) KI, the Key Identifier for an SAK
- c) LPN, Lowest Acceptable PN (least significant 32 bits for XPN Cipher Suites)
- d) Receiving, TRUE if reception for the SAK is enabled for all the receive SAs identified by AN
- e) Transmitting, TRUE if the SecY is transmitting using the SAK and its own SA with that AN
- f) Delay Protect, TRUE if LPNs are being reported sufficiently frequently to allow the recipient to provide data delay protection. If FALSE, the LPN can be reported as zero

A fixed format encoding is used. For convenience, these fields can be identified by the names and acronyms Latest AN, Old AN (LAN, OAN), Latest Key Identifier/Old Key Identifier (LKI/OKI), Lowest Acceptable PN for the Latest Key/Lowest Acceptable PN for the Old Key (LLPN/OLPN), Latest Receiving/Old Receiving (LRX/ORX), Latest Transmitting/Old Transmitting (LTX/OTX).

NOTE 1—The Latest and Old SAKs were not necessarily distributed by the same Key Server, or by the current Key Server. Both can be receiving at the same time, to enable transition from one SAK to the next without frame loss, although only one will be transmitting at any instant.

NOTE 2—When an XPN Cipher Suite is used the most significant 32 bits of the Lowest Acceptable PNs for both ANs is encoded as specified in 9.18.7.

PAEs that implement MKA Version 3 (or higher) order the MIs in the Live Peer List of transmitted MKPDUs as specified above (9.10) and encode the Key Server's transmit SA's SSCI in every MKPDU used to distribute an SAK for an XPN Cipher Suite.

9.11 Connectivity change detection

Changes in CA membership represent changes in the topology of a network that would be accompanied by link level indications, if the connectivity association represented by the CA were a LAN supported directly by media access method specific procedures, modeled by changes in the MAC_Operational and OperPointToPointMAC status parameters (IEEE Std 802.1AC, IEEE Std 802.1Q). The SecY that operates MACsec detects some of the conditions that cause such topology changes, and notifies the client of its Controlled Port through changes in the status parameters. Some of the changes that the SecY cannot detect are unimportant, and require no exceptional measures, while other information about CA membership can be accessed through the LMI to optimize the operation of certain client protocols—loss of connectivity to a Designated Router or Designated Bridge (for example) might be detected by MACsec more rapidly than by those protocols. A more significant change is the creation of new connectivity, which can only be detected by client protocols when it has occurred as opposed to when it is about to occur. New connectivity that is not signaled by MAC_Operational can cause temporary data loops in bridged networks, while a TRUE-FALSE-TRUE transition in MAC_Operational has the defined effect (for spanning tree protocols) of re-initializing state machines to deal with new connectivity.

9.12 CA formation and group CAK distribution

If the Key Server so decides, a group or pairwise CAK, can be used to distribute a group CAK and a corresponding CKN to each member of a potential group CA. Where the Key Server is the network access point and is being attached to by a number of desktop devices, the latter need have no a priori knowledge of whether they are going to be in a group CA or not. The reference table for Key Server Priority recommends a range for network access points, so they and not the desktop devices are preferentially selected as Key Server(s). The Key Server's KaY should not delete an MKA participant for a CAK that is used to distribute a further CAK until that further CAK has been used to establish peer liveness with each of the other CA members that are live peers of that distributing participant. Similarly each of the other CA members should not delete a participant used to receive a CAK until peer liveness with the Key Server has been established using the received CAK. Participants used to distribute and receive CAKs are not necessarily deleted once a CAK has been successfully deleted, but can remain active for future use.

9.12.1 Use of AES Key Wrap

Each distributed CAK shall be protected by AES Key Wrap, as specified by IETF RFC 3394. The AES Key Wrap default IV defined in that specification shall be used.

9.12.2 MKPDU application data

Each participant that considers itself to be the current Key Server and wishes to distribute a CAK encodes the following information in every MKPDU transmitted with a non-null Live Peer List:

- a) Distributed CAK, the CAK, protected by AES Key Wrap (see 6.2.4, 9.12.1, Figure 11-13)
- b) The CKN, an integral number of octets up to 32

Only one Distributed CAK is present in each MKPDU, and the MKPDU shall not contain a Distributed SAK.

NOTE 1—The Key Server will terminate the MKA Instance after a sufficient number of transmissions have ensured that the intended recipient will either have received the CAK or have been removed from the Live Peer List.

NOTE 2—The AES Key Wrap does not include the CKN (up to 32 octets) since there is no security exposure if the two are detached—the worst that can happen is that the CKN fails to locate the key.

9.13 Secure announcements

EAPOL-Announcement PDUs are not transmitted securely, but can be used to make decisions (e.g., NID selection) prior to authentication. Announcements can also be transported by MKA so that the data used to make those decisions can be retrospectively confirmed and so the participant's PAE can choose to use other authentication selections if appropriate.

9.13.1 MKPDU application data

Each participant that sends EAPOL-Announcements shall include the following Announcement parameters (if available) in each MKPDU within an MKA Announcement parameter set (Figure 11-15):

- a) The NID Set associated with the CAK for this MKA instance.
- b) The NID Set for the NID currently marked as **Access requested** (Table 11-9), if different from that associated with the CAK for this MKA instance.

Each such participant should also include the following parameters (if available), in priority order, subject to sufficient space remaining in the MKPDU for encoding the other parameters required by this specification:

- c) Global Announcement parameters, i.e., those not within the context of any NID Set.
- d) Other NID Sets.

NOTE—In contexts where a NID might be used, Global Announcement parameters can be referenced by using a null, i.e., zero length, NID name. Repeating those parameters in an announcement with a NID set with an explicit zero length name is not prohibited, but serves no useful purpose. The term 'null NID' thus refers to both Global Announcement parameters and additional parameters provided in such a NID set.

9.14 MKA participant creation and deletion

The creation of MKA protocol participants is initiated by the PAE's Logon Process (12.5) or an equivalent system process. The Key Agreement Entity (KaY) creates participants on request, and as a possible consequence of its own operation, and coordinates their operation. Each KaY shall be capable of maintaining the simultaneous operation of at least two participants for the permanently present Common Port and at least two for each virtual port.

An MKA participant shall be created when a CAK, CKN tuple (see 9.3) becomes available as a result of one of the following:

- a) Administrative configuration of a pre-shared key (PSK, 6.3.3).
- b) Successful authentication using EAP (Clause 8, 6.2.2).
- c) Receipt of a CAK, distributed by a key server, by an MKA participant (9.12).

An MKA participant can also be created, using previously cached or explicitly configured (PSK) CAK, CKN tuples as a result of the following:

- d) The SecY's or PAC's Common Port becomes operational and the Logon Process wishes to reactivate a recently active participant.
- e) Receipt of an MKPDU with a CKN that matches that for a cached CAK.
- f) Receipt of an announcement containing a desired NID, with a KMD that matches that for a cached CAK for the NID.
- g) The participant is explicitly activated by management.

An MKA participant shall be deleted as a result of any of the following:

- h) The CAK lifetime (if specified) has expired.
- i) The CAK was derived from an EAP exchange, but has not resulted in the recognition of a Live Peer (9.4) with an acceptable MACsec Capability (Table 11-6) within a period MKA Life Time (Table 9-3).

An MKA participant may be deleted as a result of any of the following:

- j) The last key server to distribute a key using that CAK is no longer in the participant's Live Peer List (9.4.3, 9.4.4) but is (as identified by its SCI) on the Live Peer List of another participant that is using the same Common Port.
- k) The number of participants would otherwise exceed the number that can be supported by the system.

9.15 MKA participant timer values

Table 9-3 summarizes each MKA participant's use of timers and their timeout values.

Table 9-3—MKA Participant timer values

Timer use	Timeout (parameter)	Timeout (seconds)
Per participant periodic transmission, initialized on each transmission, transmission on expiry (9.4).	MKA Hello Time or MKA Bounded Hello Time	2.0 0.5
Per peer lifetime, initialized when adding to or refreshing the Potential Peer List or Live Peer List, expiry cause removal from the list (9.4.3).	MKA Life Time	6.0
Participant lifetime, initialized when participant created or following receipt of an MKPDU, expiry causes participant to be deleted (9.14).		
Delay after last distributing an SAK, before the Key Server will distribute a fresh SAK following a change in the Live Peer List while the Potential Peer List is still not empty.		
Maximum suspendFor value. The maximum suspendedWhile value is MKA Life Time longer.	MKA Suspension Limit	120.0

9.16 MKA management

The PAE management process controls and monitors the operation of the KaY and MKA participants, providing access for network management through the LMI. The following variables (see also 12.2) can be used to manage the operation of the KaY for a given port (as identified by its **portNumber**, see 12.9):

- **enable:** Set by management to enable (clear to disable) the use of MKA.
- **active:** Set if there is at least one active actor, transmitting MKPDUs.
- **authenticated:** Set if the principal actor has determined that Controlled Port communication should proceed without MACsec.
- **secured:** Set if the principal actor has determined that communication should use MACsec.
- **failed:** Cleared when authenticated or secured are set, set if the latter are clear and MKA Life Time (Table 9-3) has elapsed since an MKA participant was last created.
- **actorSCI:** The SCI assigned by the system to the port (applies to all the port's actors).
- **actorPriority:** The Key Server Priority for all the port's actors.
- **keyServerSCI:** The SCI for Key Server for the principal actor. Null if there is no principal actor, or that actor has no live peers. Matches the actorSCI if the actor is the Key Server.
- **keyServerPriority:** The Key Server Priority for the Key Server for the principal actor. Matches the actorPriority if the actor is the Key Server.
- **joinGroup:** Set if the KaY will accept Group CAKs distributed by MKA.
- **formGroup:** Set if the KaY will attempt to use point-to-point CAKs to distribute a Group CAK, if it is the Key Server for the MKA instances for all the point-to-point CAKs.
- **newGroup:** Set by management if a new Group CAK is to be distributed, if the KaY is the Key Server for the MKA instances for all point-to-point CAKs. Cleared by the KaY when distribution is complete.
- **macsecCapable:** Set, for the port and applicable to all actors, by management. See 9.6.1.
- **macsecDesired:** Set, for the port and applicable to all actors, by management. See 9.6.1.
- **macsecProtect:** As used by the CP state machine, see 12.4.
- **macsecReplayProtect:** As used by the CP state machine, see 12.4.
- **macsecValidate:** As used by the CP state machine, see 12.4.
- **macsecConfidentiality:** Set by management, the confidentiality offset (9.7.1) advertised to all actors when acting as a Key Server.
- **txKN:** The Key Number (KN, 9.8) assigned by the Key Server to the SAK currently being used for transmission. Null if MACsec is not being used.
- **txAN:** The AN (9.9) assigned by the Key Server for use with txKN. Zero if MACsec is not in use.
- **rxKN:** The Key Number assigned by the Key Server to the oldest SAK currently being used for reception. The same as txKN if a single SAK is currently in use (as will most often be the case). Null if MACsec is not being used.
- **rxAN:** The AN (9.9) assigned by the Key Server for use with rxKN. The same as txAN if a single SAK is currently in use. Zero if MACsec is not in use.
- **suspendFor:** Set by management to a non-zero number of seconds between 1 and MKA Suspension Limit to initiate a suspension (9.18) of that duration (if the KaY's principal actor is the Key Server) or to request a suspension (otherwise).

- **suspendOnRequest**: Set by management to allow the KaY's principal actor to initiate a suspension if it is the Key Server and another participant has requested a suspension.
- **suspendedWhile**: Read by management to determine if a suspension is in progress and (when available) to discover the remaining duration of that suspension.

NOTE 1—Information that can be obtained from the SecY MIB is not duplicated in the above variables.

The KaY maintains the following variables for each active and potential participant:

- **CKN**: The CKN for the participant.
- **KMD**: The KMD for the participant.
- **NID**: The NID for the participant.
- **authData**: Authorization data associated with the CAK. See 6.3.
- **cached**: Set by the KaY if the participant's parameters are cached. If set, **cached** can be cleared by management to remove the participant from the cache.

The participant's CAK shall not be readable by management.

NOTE 2—Specifying authorization data is beyond the scope of this standard. Use of Radius attributes to configure Controlled Port clients is typical. See Annex D.

The following per participant variables can be used by management to monitor and control activation:

- **active**: Set if the participant is active, i.e., is currently transmitting periodic MKPDUs.
- **retain**: Set by management to retain the participant in the cache, even if the KaY would normally remove it (due to lack of use for example).
- **activate {Default, Disabled, OnOperUp, Always}**: Controls when the participant is activated. Cached entries created by the KaY as part of normal operation, without explicit management, have the value **Default**, and are activated according to the implementation dependent policies of the KaY (see 9.14). This variable can be set to any of its values by management. **Disabled** allows the cache entry to be retained, but disabled for an indefinite period. **OnOperUp** causes the participant to be activated when the PAE's Uncontrolled Port becomes MAC_Operational and when the PAE resumes following suspension (9.18). **Always** causes the participant to remain active all the time, even in the continued absence of partners. If the value is changed to **Disabled** the participant ceases operation (is deleted) immediately.

PSKs (potential MKA participants) may be created in the CAK cache by management, specifying CKN, CAK, KMD, NID, and authorization data (as required). Entries are initially created with **activate Disabled** and **retain set**. An implementation shall provide a way of disabling management creation of participants, as it offers an avenue of attack, through creation of participants with CAKs known to the attacker.

The KaY maintains the following variables for each active participant:

- **principal**: Set if the participant is currently the principal actor.
- **livePeers**: A list of the SCI's of the participant's live peers (9.4).
- **potentialPeers**: A list of the SCI's of the participant's potential peers (9.4).
- **distCKN**: The CKN for the last CAK distributed (either by the actor or one of its partners). Null if this participant has not been used to distribute a CAK.

9.17 MKA SAK distribution examples

This clause (9.17) provides some examples of MKA operation, focusing on the essentials of Key Server election and MACsec SAK distribution, installation, and use. The parts of the MKA transport component of each MKPDU—the Actor's member identifier and message number, and the member identifier and message number of each participant in the Live Peer List or Potential Peer List—and the other parameters transmitted are shown as follows:

Actor : Live Peer List : Potential Peer List: other parameters

where each MI, MN tuple is shown as X+1, X+2, etc. and tuples in the peer lists are separated by commas.

9.17.1 Two participants

Consider two stations S_A , S_B each with an MI,MN of A+..., B+... Assuming that S_A has the higher key server priority and chances to transmit first following the initialization of both participants (probably as a result of a point-to-point LAN becoming operational), the message exchange proceeds as follows:

$S_A \rightarrow A+1 :: :$ $\rightarrow S_B..$ (1.1)

$S_B \rightarrow B+1 :: A+1 :$ $\rightarrow S_A..$ (1.2)

S_A now believes that S_B is live, and will distribute an AES wrapped SAK with identifier A+1, assigning it to AN 0, and enabling S_A 's receiver and transmitter:

$S_A \rightarrow A+2 : B+1 :: \text{DistributeSAK} = \{\text{SAK}\}A+1; \text{SAKuse} = A+1.0.\text{rt}$ $\rightarrow S_B..$ (1.3)

Note that S_A is able to enable both receiver and transmitter because it was not communicating previously, so nothing is lost by adopting the new key immediately. On receipt of this message S_B will believe that S_A is live, and will install the distributed SAK, and start transmitting and receiving

$S_B \rightarrow B+2 : A+2 :: \text{SAKuse} = A+1.0.\text{rt}$ $\rightarrow S_A..$ (1.4)

If S_B , with the lower key server priority, transmits first, the exchange proceeds as follows:

$S_B \rightarrow B+1 :: :$ $\rightarrow S_A..$ (2.1)

$S_A \rightarrow A+1 :: B+1 :$ $\rightarrow S_B..$ (2.2)

$S_B \rightarrow B+2 : A+1 :: :$ $\rightarrow S_A..$ (2.3)

$S_A \rightarrow A+2 : B+2 :: \text{DistributeSAK} = \{\text{SAK}\}A+1; \text{SAKuse} = A+1.0.\text{rt}$ $\rightarrow S_B..$ (2.4)

On receipt of this fourth message, S_B can also install the key and start transmitting and receiving.

9.17.2 Another participant joins

Continuing with the prior example (9.17.1), a station S_C is attached to the LAN or powered up. Assuming that S_C is just in time to receive the last message:

$S_A \rightarrow A+2 : B+2 :: \text{DistribSAK} = \{\text{SAK}\}A+1; \text{SAKuse} = A+1.0.\text{rt}$ $\rightarrow S_B, S_C ..$ (3.4)

Then S_C will record both S_A and S_B as potential peers and send:

$S_C \rightarrow C+1 :: A+2, B+2:$ $\rightarrow S_A, S_B ..$ (3.5)

This message will prove S_C 's liveness to both S_A and S_B . S_A , acting as the Key Server, will distribute a fresh key, assigning it to AN 1 and enabling its receiver:

$S_A \rightarrow A+3 : B+2, C+1 :: \text{DistribSAK} = \{\text{SAK}\}A+2; \text{SAKuse} = A+1.0.\text{rt}, A+2.1.\text{r}$ $\rightarrow S_B, S_C ..$ (3.6)

On receiving this message S_B can install the new key, and start to receive, and can send:

$S_B \rightarrow B+3 : A+3, C+1 :: \text{SAKuse} = A+1.0.\text{rt}, A+2.1.\text{r}$ $\rightarrow S_A, S_C ..$ (3.7)

S_C also installs the new key, and starts to receive and transmit, and will send:

$$S_C \rightarrow C+2 : A+3, B+3 : : SAK_{use} = A+2.1.rt \rightarrow S_A, S_B .. (3.8)$$

Following receipt of 3.6 and 3.8 S_B can start to transmit as well as receive using the new key, as can S_A following receipt of 3.7 and 3.8. As both S_A and S_B were previously transmitting they need to know that all active participants can receive using the new key, before they use it to transmit.

9.18 In-service upgrades

The control plane software of a system that is a member of a CA can be upgraded, temporarily suspending MKA operation, without interrupting the secure data connectivity provided by the CA, if the system and each of its peers in the CA support in-service upgrades (5.11.4). An SAK that is already in use will continue to be used provided that each of these peers does not conclude that it is the CA's only active member, and provided that a fresh SAK is not distributed by a recognized Key Server. If each member's `suspendedWhile` timer is not zero these conditions will be met (12.2, 12.4, 12.5).

NOTE—A KaY's `suspendedWhile` timer takes a non-zero value to indicate a suspension, a period during which one or more of its principal actor's partners has ceased (or can be expected to cease) transmitting MKPDUs.

This standard specifies procedures that allow a Key Server to control the initiation, proposed duration, and possible early termination of any MKA suspension by communicating the current value of the `suspendedWhile` timer in MKPDUs. A Key Server that supports in-service upgrades always includes the value in each MKPDU transmitted. A suspension is terminated by expiry of a participant's `suspendedWhile` timer, by the Key Server communicating a zero value, or by a Key Server that does not support in-service upgrades omitting the value from transmitted MKPDUs.

9.18.1 Initiating suspension

A participant other than the Key Server can request a suspension by transmitting a (non-zero) `suspendFor` value in an MKPDU. The participant can maintain the request for an indefinite period by repeating the value in each MKPDU transmitted before it suspends MKA operation. If the request is no longer relevant, perhaps because the participant has upgraded its control plane software without waiting for a suspension, the participant can drop the request by transmitting MKPDUs with a `suspendFor` value of zero.

The Key Server records, for each of the participants on its Live Peer List, the lesser of the values of the `suspendFor` parameter in the last MKPDU received and the MKA Suspension Limit specified in Table 9-3.

A Key Server should not initiate a suspension until it has started transmitting using the last SAK it has distributed and is no longer receiving using any prior SAK. This requirement is equivalent to stating that the CP state machine should be in state RETIRE having completed the actions on entry to that state (see Figure 12-2). If these conditions are satisfied, and the value of the Key Server's own `suspendFor` parameter is non-zero or the Key Server's policy control `suspendOnRequest` is True and one of the received `suspendFor` parameters is non-zero, the Key Server will initiate a suspension. The Key Server can apply additional policy controls on the setting of its `suspendFor` and `suspendOnRequest` parameters to limit when a suspension can occur, or to limit the frequency of suspensions. Any participant can also apply its own policy controls, limiting (for example) how long it is prepared to wait for the Key Server to initiate a suspension of adequate duration.

To initiate a suspension the Key Server sets the value of its `suspendedWhile` timer to the greatest of the applicable `suspendFor` values. Subsequently `suspendedWhile` is decremented once per second, and then set to the greater of its new value and the greatest applicable `suspendFor` value. The value of `suspendedWhile` (decrementing over time) is transmitted in all MKPDUs for all MKA instances that have elected it Key Server. These transmissions should persist for at least MKA Life Time before the Key Server suspends its own operation (if desired).

A participant, other than the Key Server, that wishes to suspend its own operation of MKA includes a non-zero **suspendFor** value in all MKPDUs transmitted and should not suspend its operation of MKA until it receives an MKPDU from the Key Server with a non-zero **suspendedWhile** value that is greater than or equal to the value of its **suspendFor** parameter.

NOTE—Details of MKA operation are likely just a part of the concerns to be addressed when performing a system software upgrade, and the overall considerations will not necessarily allow the KaY to delay suspension to facilitate transmission and reception of MKPDUs as recommended. The probability that the transmission of a single MKPDU by the participant requesting suspension will be successfully received and acted on by the Key Server might be acceptable, as might the probability of reception of a single MKPDU transmitted by the Key Server.

If the requesting participant does not suspend its own operation of MKA, but continues to transmit a non-zero **suspendFor** value in subsequent MKPDUs, then the Key Server's **suspendedWhile** value will continue to be at least that of the requested value—provided that the Key Server does not suspend its own operation and its **suspendOnRequest** parameter remains True. The MKA Suspension Limit specified in Table 9-3 limits the time for which a participant will maintain existing connectivity after all partners requesting suspension have been removed from its Live Peer List, on the assumption that these partners have been suspended and will resume operation.

9.18.2 Suspending

A CA member should not suspend its own operation, unless it is

- a) Receiving and transmitting using a single SAK, or
- b) Constrained by policy controls that place limits on the time that it is prepared to wait to suspend.

9.18.3 Suspended members

A CA member that suspends MKA operation while using MACsec to secure connectivity continues to generate, transmit, receive, and verify secure data frames as specified by IEEE Std 802.1AE, maintaining the receive and transmit SAs current at the time of the suspension. In effect, the CP state machine remains in state RETIRE.

A suspended member is not required to retain any record of its MI, or of its Live Peer List or Potential Peer List, for any of its participants. It requires a record of any SAK(s) used by transmit and receive SAs only in so far as is required by the operation of those SAs, and shall not redistribute or otherwise share the SAK(s) with other participants while suspended or at any later time.

A suspended member does require access, when operation is resumed, to the following information required by the operation of each SA in use:

- a) The SCI
- b) The AN

A suspended member is not required to retain the KI of any SAK in use, but can report a zero value (after resuming operation) when responding to management requests and completing KI fields in MKPDUs. The rules for SAK generation (9.8) ensure that a fresh SAK will be distributed after a suspended member resumes operation with a new MI.

A member can suspend MKA operation because part of its system's functionality will not be available during an in-service upgrade. The member might not be able to transmit and receive frames because software associated with a particular interface module is to be upgraded, for example. The suspended member can retain its MI provided that it continues to operate MKA, with the exception that any MKPDUs that might be generated for transmission or that are received can be discarded. The normal operation of MKA will result in the removal of partners from the Live Peer List and Potential Peer List (9.4.3) if MKPDU loss persists.

9.18.4 Resuming operation

When a CA member resumes, it sets its own **suspendedWhile** timer value to Max Suspension Limit or to some lower policy determined limit. It also sets its **suspendFor** parameter value to zero once it has determined that the upgrade has completed successfully (see 9.18.6).

NOTE 1—The Max Suspension Limit is specified in Table 9-3.

The CA member might have suspended only because it was unable (for the duration of the suspension) to receive or transmit MKPDUs or install fresh SAKs. If it has continued to operate MKA while suspended, its MI, Live Peer List, and Potential Peer List will be retained when it resumes. Otherwise, i.e., if MKA operation ceased, it will select a fresh MI and its Live Peer List and Potential Peer List will be initialized and empty on resumption.

If the Key Server that initiated the suspension suspends itself, then it is possible that a participant for another CA member (that might or might not have suspended itself) will be elected Key Server before the initiating Key Server resumes. If the newly elected Key Server's **suspendedWhile** timer is running, then it will distribute the (decrementing) value of that timer and thus prolong the suspension. This provision means that it is unnecessary for a resuming member to remember whether it was or was not the Key Server prior to being suspended. If the initiating Key Server resumes operation before the timer expires it will once more assume responsibility for monitoring the suspension.

The current Key Server shall terminate the suspension before the value of its **suspendedWhile** timer reaches zero by resetting that value (included in all transmitted MKPDUs) to zero under the following conditions:

- a) Either
 - 1) For every active receive SA it has a live peer with an SCI that matches that of the receive SA;
or
 - 2) It has a live peer with an SCI that does not match any existing receive SA;and
- b) It has recorded a value of zero for the **suspendFor** parameter received from each of these live peers.

To determine, for the purpose of test a)1), that a receive SA is active the Key Server monitors the value of the InPktsOK management counter (see Figure 10-5 and 10.7.9 of IEEE Std 802.1AE-2018) for the SA. The receive SA is not active if the counter has not been incremented for MKA Life Time. Test a)2) detects the addition of a new CA member.

NOTE 2—For the common case of a point-to-point CA, these conditions simplify to either having a live peer communicating a zero **suspendFor** value or not having received any secured frames for MKA Life Time.

NOTE 3—While the receipt of validated secured data frames does not ensure that communication is with a live peer in current possession of a shared SAK, the lack of any such reception for a period of MKA Life Time (6 seconds) is a strong practical indication of the absence of such a peer.

9.18.5 XPN support

If a receiver loses more than 2^{30} consecutive frames when an XPN capable Cipher Suite is being used, the 32 most significant bits of the PN of the next frame to be received might be recovered incorrectly. If more than 2^{32} frames are lost, these most significant bits will be recovered incorrectly. All subsequent secured frames will fail validation and be discarded, unless some means other than the receipt of secured data frames is used to determine their value.

When a suspension is in progress, the risk of losing a large number of frames is increased as the operation of supervisory protocols that would otherwise detect temporary loss of connectivity might also be suspended.

At 100 Gb/s 2^{30} minimum sized secured frames can be transmitted in 10 seconds, well within the potential duration of a suspension. To ensure that the most significant bits are recovered when all CA members resume MKA operation, the most significant 32 bits of the Lowest Acceptable PN for the Latest Key and the Old Key are communicated when in-service upgrades are supported and any XPN capable Cipher Suite is being used.

9.18.6 Managing in-service upgrades

Careful planning is required when upgrading systems that compose a network if the network is to remain in operation, and if the costs of recovering from a failed upgrade are considerable. Best practices include the following:

- a) Ensuring that the network manager has an up to date record of all the systems in the network, and of the network configuration.
- b) Ensuring that the network manager has an up to date and independently backed up record of the software, software revision levels, and configuration parameters currently used by the systems to be upgraded, by their neighbors, and by their other peers in the network.
- c) Off-line verification and testing for compatibility between the proposed new software and configuration parameters and the existing software and configuration of neighbors and other peers.
- d) Retention of the existing software and configuration parameters by the systems being upgraded until successful operation with the new software and parameters is confirmed.
- e) Use of a ‘dead man’ timer by the system to be upgraded, so that the system will automatically revert to the prior software and configuration if satisfactory management communication cannot be established with the network operations center after the upgrade.
- f) Off-line verification and testing of any such fallback mechanism.
- g) Whenever possible, upgrading only one intermediate system (bridge or router) at a time, confirming the success of the upgrade before upgrading additional systems.

In providing continued secure data connectivity while an in-service upgrade is performed, MKA makes a modest contribution to the task of upgrading network systems. MKA lacks the knowledge and scope to ensure or to check that best practices are being followed when the upgrade is being performed, and the network administrator is not relieved of these responsibilities.

The maximum time allowed for suspension, 120 seconds, is believed to be adequate for an upgrade, followed by expiry of the dead man timer, and reversion to the original software revision and configuration. However, if MKA successfully resumes operation, it could begin to distribute a new SAK at a time when the network operation center might not (for reasons completely unrelated to the use of MKA or MACsec) have re-established management connectivity with the upgrading system. If the system is then unilaterally suspended by operation of the dead man timer, it is possible that the members of the CAK will not all converge on use of the latest, or the old, SAK. The Key Server shall not redistribute the previous SAK (9.8). As a consequence, if a dead man timer or similar mechanism is being used by a resuming system, that system should not reset its `suspendFor` parameter to zero (see 9.18.4, 9.8) until the dead man timer has been reset.

MKA’s in-service upgrade support can be deployed in environments where a comprehensive approach to system upgrade is already in place, and already synchronizes update and suspension activities. Such an existing approach can suspend MKA operation, as required, by coordinating the setting of `suspendFor` and `suspendOnRequest` parameters using each system’s LMI. The values of `suspendedWhile` parameters for both Key Servers and other participants may also be set directly using the LMI, thus avoiding adding additional protocol dependencies to the existing coordination mechanism. Any such directly set values shall be consistent with the values that MKA would, and if not suspended will, communicate.

If the **suspendFor** or **suspendedWhile** timer values are set (either by using the MIB specified in Clause 13, or through the operation of other protocols) when there is no need for a suspension (i.e., the conditions for terminating the suspension are already satisfied) the suspension is terminated immediately and the values reset prior to including the timer value in any subsequent MKPDU. The conditions for such an immediate termination could occur as a result of one CA member initiating a suspension after other members have downloaded new software that enables them to upgrade during the suspension but before their own timer values are set.

9.18.7 MKPDU application data

Each participant that is capable of supporting in-service upgrades shall include the following parameter in each MKPDU transmitted (see Figure 11-16):

- a) MKA suspension time.
The value transmitted is that of the **suspendedWhile** timer if the sending participant considers itself to be the Key Server for the MKA instance (i.e., has set bit 8 in octet 3 of the Basic Parameter Set, see Figure 11-8), and is the value of the **suspendFor** parameter otherwise.

and, when the Current Cipher Suite uses extended packet numbering, the following parameters:

- b) The 32 most significant bits of the Lowest Acceptable PN for the Latest Key.
- c) The 32 most significant bits of the Lowest Acceptable PN for the Old Key (if in use).

A receiving participant sets its member's **suspendedWhile** timer to a received **suspendedWhile** value iff it is the member's principal actor and agrees that the transmitter is the Key Server for its MKA instance.

A receiving participant records a **suspendFor** value received from any live partner, superseding any prior **suspendFor** value received from that partner.

9.19 In-service upgrade examples

This subclause (9.19) provides some examples of MKA operation, focusing on suspension. The parts of the MKA transport component of each MKPDU—the actor's member identifier and message number, and the member identifier and message number of each participant in the Live Peer List or Potential Peer List—and (some of) the other parameters transmitted are shown, as in 9.17, as follows:

Actor : Live Peer List : Potential Peer List: other parameters

where each MI, MN tuple is shown as X+1, X+2, etc. Initial MN values in these examples are arbitrary. Tuples in the peer lists, and other parameters, are separated by semi-colons.

9.19.1 Requested by end station in point-to-point CA

An end station port **S**, that is not the Key Server, requests suspension from the Key Server, **K**, by sending an MKPDU with a **suspendFor** value (encoded in the MKA Suspension Time field) of 60 seconds. **K** has **suspendOnRequest** True, and responds with an MKPDU with a **suspendedWhile** value (again encoded in the MKA Suspension Time field) of 60 seconds. On receipt, **S** suspends itself.

S_A	$\rightarrow A+47:F+63::suspendFor = 60$	$\rightarrow$	$K_F..$ (1.1)
K_F	$\rightarrow F+64:A+47::suspendedWhile = 60$	$\rightarrow$	$S_A..$ (1.2)
S_A	suspends		.. (1.3)

The next two or three periodic transmissions by **K** occur while **S** (as **A**) is still on K_F 's Live Peer List, so the value of **suspendedWhile** remains at 60 seconds. The value of **suspendedWhile** in subsequent transmissions is decremented over time, but before it reaches zero (having reached 30 seconds in this

example) S resumes with a new MI R. S has not kept accurate track of the duration of the suspension, and simply assumes a **suspendedWhile** value of 120 seconds, the maximum that can be requested.

$K_F \rightarrow F+65:A+47::suspendedWhile = 60 \rightarrow \dots$ (1.4)

$K_F \rightarrow F+66:A+47::suspendedWhile = 60 \rightarrow \dots$ (1.5)

$K_F \rightarrow F+67::suspendedWhile = 58 \rightarrow \dots$ (1.6)

...

$K_F \rightarrow F+91:suspendedWhile = 30: \rightarrow \dots$ (1.7)

S_R resumes, assuming **suspendedWhile** = 120 .. (1.8)

S then exchanges MKPDUs with K, and is recognized as a live peer. K terminates the suspension, and distributes a fresh SAK, with the key identifier (in this example) of F+2.

$S_R \rightarrow R+1::suspendFor = 0 \rightarrow K_F..$ (1.9)

$K_F \rightarrow F+92::R+1:suspendedWhile = 30 \rightarrow S_R..$ (1.10)

$S_R \rightarrow R+2:F+92::suspendFor = 0 \rightarrow K_F..$ (1.9)

$K_F \rightarrow F+93:R+2::suspendedWhile = 0; DistribSAK = \{SAK\}F+2;$
 $SAKuse = F+1.0.rt, F+2.1.r \rightarrow S_R..$ (1.10)

S can then install and use the fresh key, exchanging MKPDUs as required (see 9.17 for relevant examples).

9.19.2 Initiated by Key Server in point-to-point CA

A Key Server initiates the suspension by sending an MKPDU with a **suspendedWhile** value (encoded, as always, in the MKA Suspension Time field) of 60 seconds, before suspending. In this particular example the other CA member takes the opportunity of suspending and upgrading itself at the same time.

$K_F \rightarrow F+64:A+47::suspendedWhile = 60 \rightarrow S_A..$ (2.1)

S_A suspends .. (2.2)

$K_F \rightarrow F+65:A+47::suspendedWhile = 60 \rightarrow \dots$ (2.3)

$K_F \rightarrow F+66:A+47::suspendedWhile = 60 \rightarrow \dots$ (2.4)

K_F suspends .. (2.5)

K resumes, assuming an MI of D, and a **suspendedWhile** value of 120 seconds.

K_D resumes, assuming **suspendedWhile** = 120 .. (2.6)

$K_D \rightarrow D+1::suspendedWhile = 120 \rightarrow \dots$ (2.7)

$K_D \rightarrow D+2::suspendedWhile = 118 \rightarrow \dots$ (2.8)

S resumes, assuming an MI of G, and a **suspendedWhile** value of 120 seconds.

S_G resumes, assuming **suspendedWhile** = 120 .. (2.9)

$S_G \rightarrow G+1::suspendFor = 0 \rightarrow K_D..$ (2.10)

$K_D \rightarrow D+3::G+1:suspendedWhile = 118 \rightarrow S_G..$ (2.11)

$S_G \rightarrow G+2:D+3::suspendFor = 0 \rightarrow K_D..$ (2.12)

$K_D \rightarrow D+4:G+2::suspendedWhile = 0; DistribSAK = \{SAK\}D+1;$
 $SAKuse = 0.0.rt, D+1.1.r \rightarrow S_G..$ (2.13)

The installation and use of the fresh key now proceeds as before.

9.19.3 Intermediate systems suspending multiple CAs

An intermediate system, a router or bridge, will usually have to suspend MKA operation in the multiple CAs that it connects, as a control plane software upgrade will affect all of its ports. It might be the Key Server for some CAs and not for others. Careful planning is required (see 9.18.6) when upgrading intermediate systems in a network, and there are limits to the assistance and safeguards that can be provided by the operation of a local protocol, such as MKA. However, MKA, as shown in this deliberately complex example, does provide some help when multiple systems are involved and there has been a lack of coordination.

In this example, an intermediate system I has ports 1, 2, 3 with MKA participants I1, I2, I3, and neighbors A, B, C, respectively. I2 is already participating in a suspension initiated by B, which has suspended itself. The network administrator instructs I to upgrade, causing **suspendFor** to be set 60 seconds on each port. I1 is the Key Server for its CA, and can set and start transmitting **suspendedWhile** immediately. However it still has to arrange to suspend operation on its other ports, so it cannot suspend I1 immediately, so **suspendFor** and (as a consequence) **suspendedWhile** for I1 will remain at 60 seconds for the time being. I2 is also (in B's absence) the Key Server for its CA, and can also set **suspendedWhile** directly. C, rather than I3, is the Key Server for the third port's CA.

I2_P **suspendedWhile** = 15 .. (3.1)

I1 **suspendFor** = 60, I2 **suspendFor** = 60, I3 **suspendFor** = 60 .. (3.2)

I1_J → J+31:V+60::**suspendedWhile** = 60 → A_V.. (3.3)

I2_P → P+29:R+58::**suspendedWhile** = 60 → B_R.. (3.4)

I3_Q → Q+33:W+62::**suspendFor** = 60 → C_W.. (3.5)

I might have to persist with these transmissions for some time, but if (and as soon as) C has **suspendOnRequest** set it will respond, and this can happen immediately.

C_W → W+63:Q+33::**suspendedWhile** = 60 → I3_Q.. (3.6)

I can then suspend and upgrade. While it is still suspended B might resume, but its newly assumed **suspendedWhile** value of 120 seconds will provide sufficient time for I2's suspension (B was not operating when I2 suspended so has no information that would allow it to adopt a lower value).

B_K → K+1:::**suspendedWhile** = 120 → .. (3.7)

When I resumes, the participants for each of its ports and their partners will exchange MKPDUs as usual in order to recognize live peers, and fresh keys will be distributed.

9.19.4 Key Server suspends in a group CA

A, B, and C are members of a group CA. A, the Key Server, is to suspend to allow its control plane to be upgraded, and 30 seconds is believed to be sufficient for this to occur. The process is started by setting A's **suspendFor** parameter. This causes A to set its **suspendedWhile** parameter, and to transmit periodic MKPDUs for the next MKA Life Time (6 seconds) before suspending.

A_E → E+14:Y+15,L+14::**suspendedWhile** = 30 → B_Y, C_L.. (4.1)

For the following MKA Life Time A will remain on B and C's Live Peer Lists, so neither will claim to be the new Key Server. Finally, assuming that B has the higher Key Server Priority, B will become the Key Server and transmit the, by now decremented and continually decrementing, value of **suspendedWhile** in its periodic transmissions.

B_Y → Y+21:L+20::**suspendedWhile** = 24 → B_Y, C_L.. (4.1)

When A resumes, it will advertise **suspendedWhile** as 120 seconds, before exchanging MKPDUs and realizing that the conditions for terminating the suspension have been met. However, if A does not resume for any reason, B will terminate the suspension in 24 seconds.